

Veniamin Velikoretskikh

veniamin@pdx.edu

CS410 computer vision and deep learning assignment 3

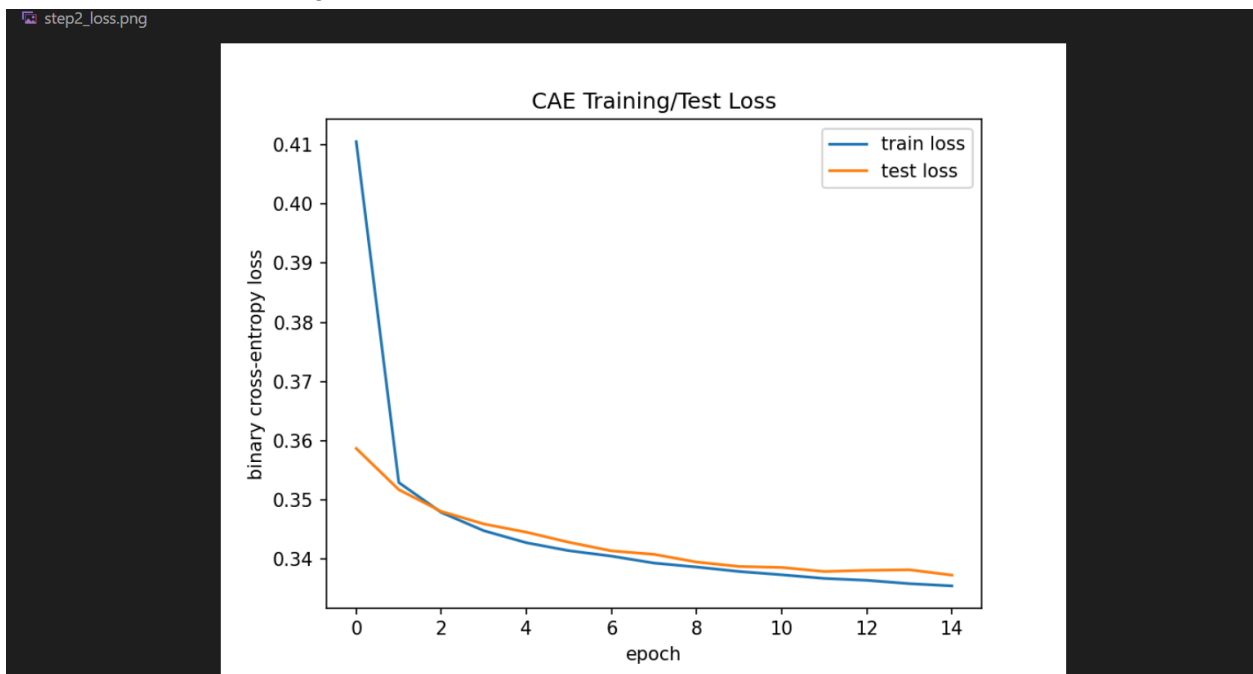
For this programming assignment, I decided to go with the first option, to design and train a convolutional autoencoder, also known as CAE. This encoder was trained on the Fashion-MNIST dataset. It compressed each 28-by-28 grayscale image down to 2 numbers. Then, in the decoder, it reconstructed the image back to its original size of 28 by 28.

Architecture - step 1

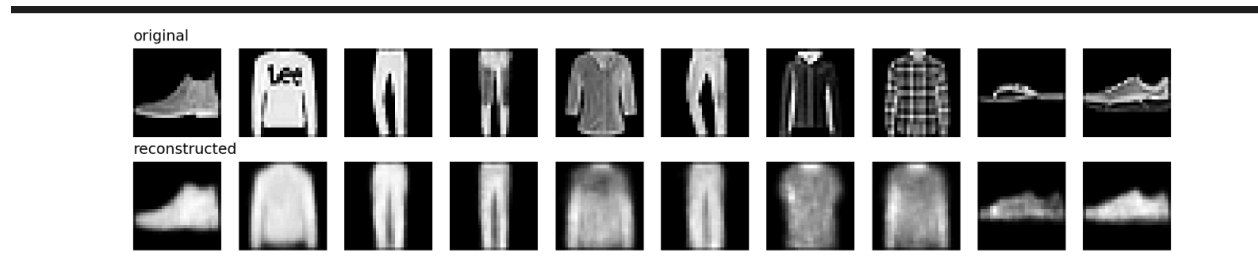
The encoder has 3 layers: 32, 16, and 8 filters with max-pooling between each. This reduces the 28 by 28 input down to a 7 by 7 by 8 feature map. Then this map is passed through a Dense bottleneck layer. After that, the decoder expands the feature map back to its original size of 28 by 28, passing through 3 filters again.

Training the CAE - step 2

The training was done with an Adam optimizer and binary cross-entropy loss. It was computed between each input image and what was reconstructed. This was done for 15 epochs.



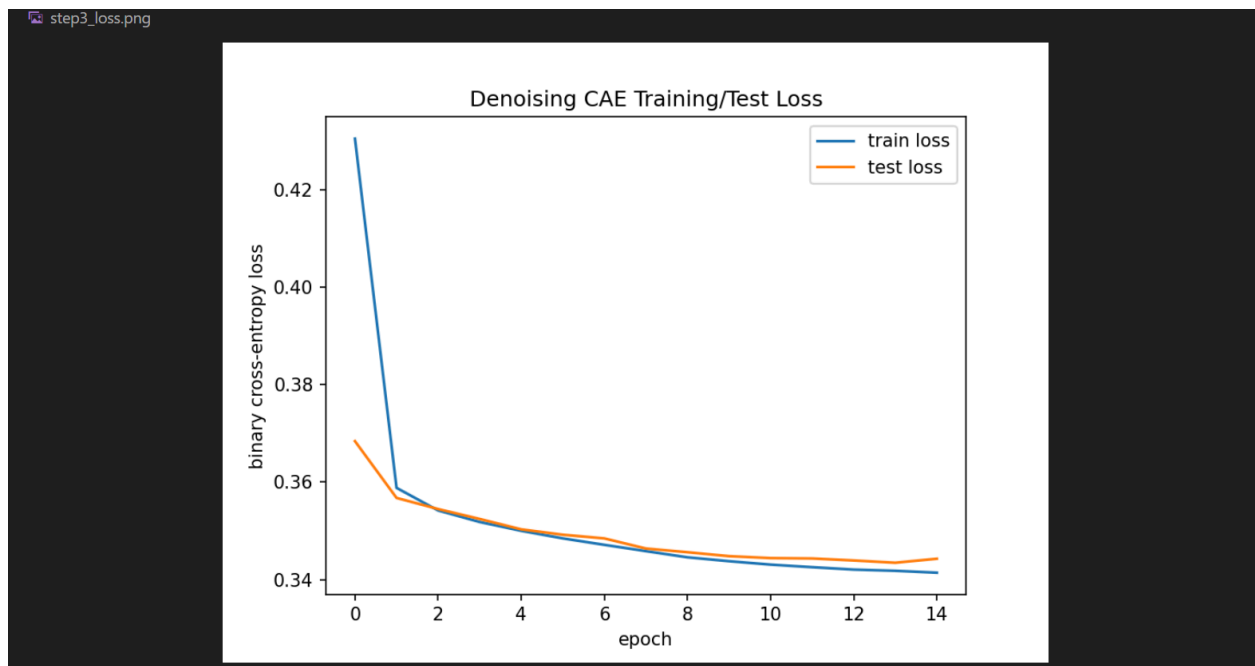
The loss was decreasing smoothly and consistently through the training. The test loss remained close to training, meaning the model was learning stably and wasn't overfitting. The curve started to flatten because the model was reaching its limits on what a 2d bottle neck can represent for this dataset.



The reconstructed images only capture the coarse shape of the original image. It does lose some of the fine detail. I think this is to be expected because the images were compressed down to 2 numbers.

Denoising CAE - step 3

The same autoencoder architecture was used for the denoising part. Gaussian noise was added to the input images; this trains the network to remove the noise rather than simply copying the input. At first I tried to use a noise factor of 1, but that was too much, and the reconstructed images were not clearly shown. A person couldn't say what it is. I later went down to a noise factor of 0.5, and it worked a lot better.



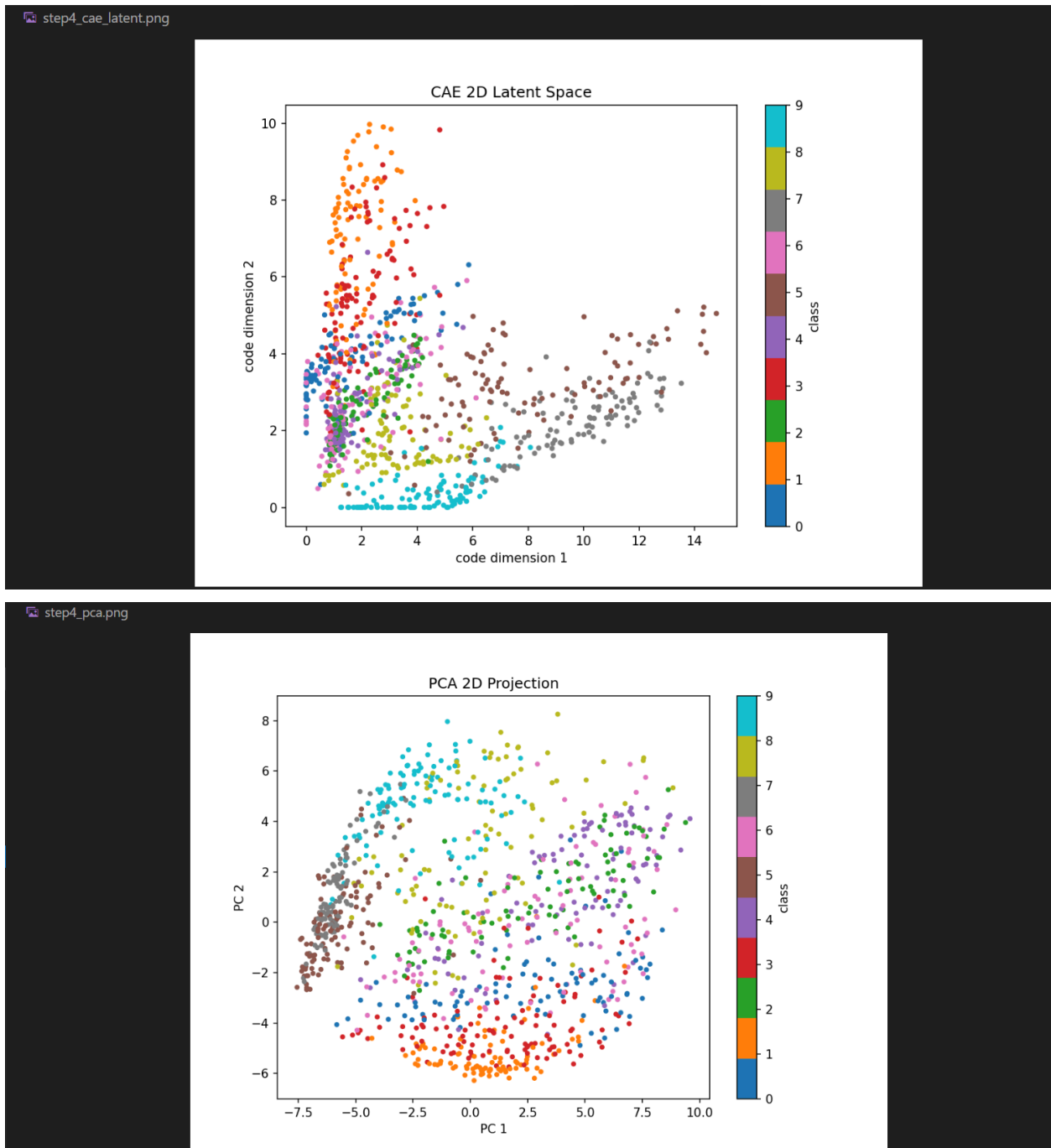
Just like before, the loss was decreasing smoothly and consistently through training and test loss. The train and test loss are close together and show little to no divergence.



With the noise factor reduced to 0.5, the results increased significantly. After the noise is added, the denoised images clearly show the shape of the original image. Not capturing every detailed feature, but captures the overall image and features.

Latent space and PCA - step 4

Used the encoder to extract 2-number codes for the 1000 randomly sampled test images. Then plotted with color for each class. The same was plotted with PCA to compare.



Both methods that I used gave me some meaningful but still not perfect separation of the classes. With the CAE, I've noticed that the classes overlap more closely with the origin. The

PCA shows a more spread-out class separation and has less overlap than the CAE. What I noticed is that the PCA has classes and data plots in the negative section of the grid. Compared to the CAE, I don't have that. I think that's because of the ReLU activation, that doesn't give me negative outputs. If I were going to go deeper into this, I would change the structure of the autoencoder, starting with changing the activation function. But for a 2 dimension the CAE, doesn't perform bad.